

PSD

System Wykrywania Fraudów Kartowych

Wykonał

Julian Żebrecki 330230

Jakub Jażdżyk 318663

PSD – System Wykrywania Fraudów Kartowych

1. Wprowadzenie

Celem projektu było opracowanie kompletnego systemu umożliwiającego wykrywanie anomalii w transakcjach wykonywanych kartami płatniczymi. Rozwiązanie zostało zaprojektowane zgodnie z wymaganiami przedmiotu Przetwarzanie Strumieniowe Danych i wykorzystuje architekturę opartą na przetwarzaniu zdarzeń w czasie rzeczywistym.

System analizuje strumień transakcji generowanych przez symulator kart płatniczych, wykrywa potencjalne nadużycia finansowe oraz przekazuje informacje o wykrytych zagrożeniach do dedykowanego kanału alarmowego. W projekcie wykorzystano technologie Apache Kafka, Apache Flink oraz MongoDB, które wspólnie tworzą skalowalne środowisko do przetwarzania danych strumieniowych.

W ramach projektu przygotowano również narzędzia umożliwiające monitorowanie działania systemu, wizualizację generowanych danych oraz prezentację wykrytych alertów.

Kluczowe parametry systemu

10 000 Kart płatniczych	8 000 Użytkowników	5–10 Transakcji / sek	5% Szansa anomalii
3× Mnożnik EMA	500 km/h próg podróży	10 s Min. odstęp tx	60 s Checkpoint Flink

Stos technologiczny

Komponent	Technologia	Wersja	Rola
Przetwarzanie strumieniowe	Apache Flink (PyFlink)	1.19	Detekcja anomalii, zarządzanie stanem
Message broker	Apache Kafka (Confluent)	7.6.1	Komunikacja między komponentami
Koordinacja Kafka	Apache ZooKeeper	7.6.1	Zarządzanie klastrem Kafka
Baza danych alertów	MongoDB	7	Persystencja alertów fraudowych
Język aplikacji	Python	3.11	Wszystkie komponenty aplikacyjne
Lib terminalna	Rich	≥ 13.7.0	Wizualizacja w terminalu (live table, banery)
Kafka client	confluent-kafka	≥ 2.3.0	Producent / konsument Python
Konteneryzacja	Docker + Docker Compose	—	Orkiestracja całej infrastruktury
UI Kafka	provectuslabs/kafka-ui	0.7.2	Przeglądarka tematów i konsumentów
UI MongoDB	mongo-express	1.0.2-20	Przeglądarka kolekcji fraud_alerts

2. Architektura rozwiązania

Architektura systemu została zaprojektowana zgodnie z podejściem Event-Driven Architecture. Każda transakcja kartowa traktowana jest jako niezależne zdarzenie, które przepływa przez kolejne etapy przetwarzania.

Proces rozpoczyna się od działania generatora danych, który symuluje zachowanie użytkowników kart płatniczych. Wygenerowane transakcje są publikowane do tematu Kafka „transactions”. Następnie aplikacja Apache Flink odbiera dane, utrzymuje stan każdej karty oraz wykonuje analizę anomalii. W

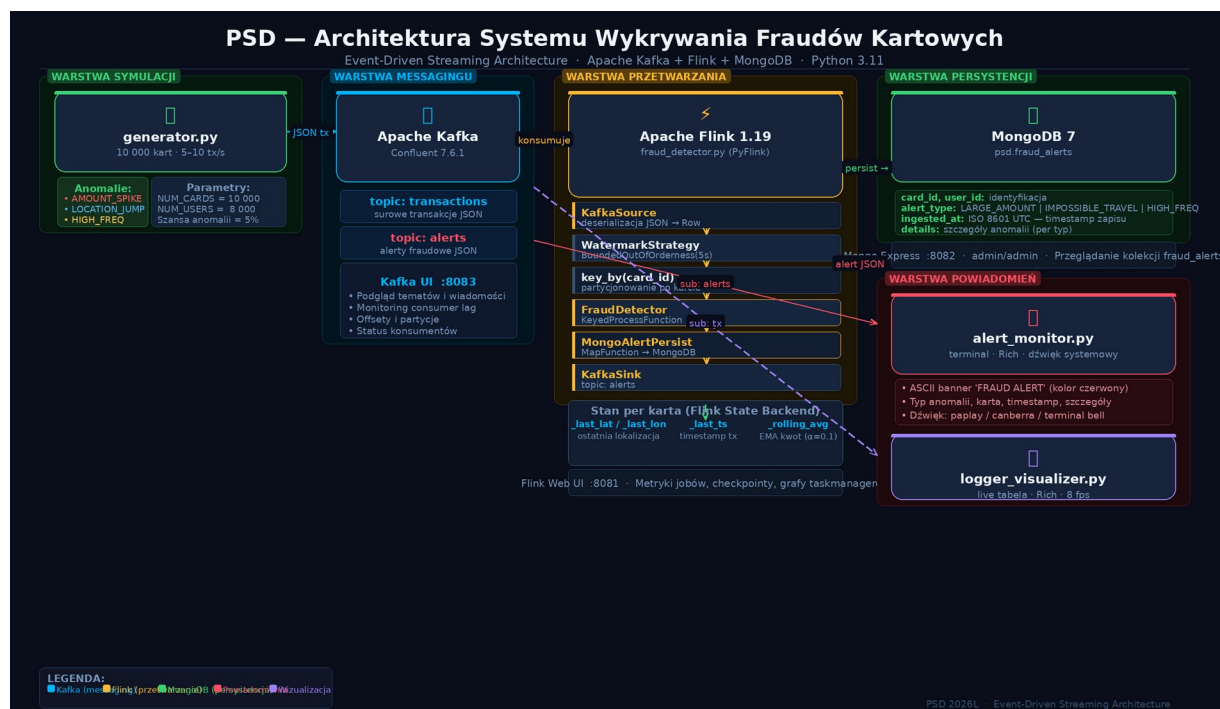
przypadku wykrycia podejrzanego zdarzenia tworzony jest alert, który zostaje zapisany w bazie MongoDB oraz wysłany do tematu Kafka „alerts”.

Ostatnim elementem systemu jest aplikacja monitorująca, która odbiera komunikaty alarmowe i prezentuje je operatorowi w czytelnej formie tekstowej.

Takie podejście pozwala na rozdzielenie odpowiedzialności pomiędzy komponentami, zwiększa odporność systemu na awarie oraz umożliwia łatwe skalowanie poszczególnych elementów.

Schemat przepływu danych:

Generator transakcji → Kafka → Flink Fraud Detector → Kafka Alerts → Monitor Alertów
↓
MongoDB



Rysunek 1. Schemat systemu PSD — przepływ danych od generatora przez Kafka i Flink do warstwy persystencji i powiadomień.

Warstwy architektury

Warstwa	Komponent	Opis
Symulacja	generator.py	10 000 kart, 8 000 userów, 5–10 tx/s, 5% szansa

Warstwa	Komponent	Opis
		anomalii
Messaging	Apache Kafka	Dwa tematy: transactions (surowe tx) i alerts (fraudy)
Przetwarzanie	Apache Flink	PyFlink KeyedProcessFunction - stan per karta, 3 reguły detekcji
Persystencja	MongoDB	Kolekcja psd.fraud_alerts z timestampem ingested_at
Powiadomienia	alert_monitor.py	ASCII banner, Rich panel, dźwięk systemowy
Wizualizacja	logger_visualizer.py	Live tabela transakcji, bufor 20 wpisów, 8 fps
Observability	Flink UI / Kafka UI / Mongo Express	Web UIs: :8081 / :8083 / :8082

Interfejsy webowe (Observability)

Usługa	URL	Dane dostępne	Funkcje
Flink Web UI	:8081	—	Metryki jobów, grafy taskmanagerów, status checkpointów
Mongo Express	:8082	admin / admin	Przeglądanie i filtrowanie kolekcji fraud_alerts
Kafka UI	:8083	—	Tematy, konsumenci, offsety, lag, consumer groups

3. Opis komponentów systemu

Generator transakcji

Generator odpowiada za tworzenie realistycznego strumienia operacji kartowych. W systemie symulowanych jest 10 000 kart płatniczych przypisanych do 8 000 użytkowników. Każda karta posiada własne parametry, takie jak średnia wartość transakcji, limit wydatków oraz lokalizacja bazowa.

Generowane dane zawierają identyfikator karty, identyfikator użytkownika, współrzędne GPS, wartość transakcji, limit karty oraz znacznik czasu. Wszystkie komunikaty przesyłane są w formacie JSON.

W celu przetestowania skuteczności algorytmów detekcji generator okresowo wprowadza sztuczne anomalie. Symulowane są trzy podstawowe rodzaje nieprawidłowości:

- Nagły wzrost wartości transakcji.
- Nagła zmiana lokalizacji wykonania płatności.
- Bardzo wysoka częstotliwość wykonywania operacji.

Dzięki temu możliwe jest zweryfikowanie poprawności działania całego systemu bez konieczności korzystania z rzeczywistych danych bankowych.

Detektor fraudów

Najważniejszym elementem projektu jest aplikacja Apache Flink odpowiedzialna za analizę danych w czasie rzeczywistym. Dane są grupowane według identyfikatora karty, dzięki czemu każda karta posiada własny, niezależny stan przechowywany w pamięci Flinka.

Podczas przetwarzania system zapamiętuje między innymi ostatnią lokalizację użytkownika, czas wykonania poprzedniej transakcji oraz średnią wartość operacji wyliczaną metodą wykładniczej średniej kroczącej (EMA).

Dla każdej nowej transakcji wykonywane są reguły detekcji anomalii. Jeżeli którakolwiek z nich zostanie spełniona, generowany jest alert zawierający szczegóły wykrytego zdarzenia.

Monitor alertów

Monitor alertów pełni funkcję centrum obserwacyjnego systemu. Odbiera komunikaty alarmowe z Kafki i prezentuje je operatorowi w czasie rzeczywistym. Dla każdego alertu wyświetlane są szczegóły dotyczące rodzaju zagrożenia, identyfikatora karty oraz parametrów, które spowodowały uruchomienie alarmu.

Wizualizator transakcji

Wizualizator umożliwia podgląd bieżącego strumienia danych. Narzędzie prezentuje ostatnio wygenerowane transakcje w postaci tabeli aktualizowanej na żywo. Dzięki temu możliwe jest szybkie sprawdzenie poprawności działania generatora oraz obserwacja przepływu danych przez system.

4. Algorytmy wykrywania anomalii

W projekcie zastosowano trzy niezależne algorytmy wykrywania nieprawidłowości.

Pierwszy mechanizm analizuje wartość transakcji względem historycznych zachowań użytkownika. System oblicza wykładniczą średnią kroczącą wydatków dla każdej karty. Jeżeli nowa operacja znacząco przekracza typowy poziom wydatków, generowany jest alert typu `LARGE_AMOUNT`.

Drugi algorytm wykrywa niemożliwe podróże. Dla kolejnych transakcji tej samej karty obliczana jest odległość geograficzna oraz teoretyczna prędkość przemieszczania się użytkownika. Jeżeli wyliczona prędkość przekracza ustalony próg 500 km/h, system zakłada możliwość oszustwa i generuje alert `IMPOSSIBLE_TRAVEL`.

Trzecia metoda koncentruje się na analizie częstotliwości operacji. Jeżeli kolejne płatności wykonywane są w bardzo krótkich odstępach czasu, może to świadczyć o automatycznych próbach wykorzystania danych karty. W takim przypadku tworzony jest alert `HIGH_FREQUENCY`.

Połączenie trzech różnych metod pozwala wykrywać zarówno proste próby oszustw, jak i bardziej złożone anomalie zachowań użytkowników.

Typy wstrzykiwanych anomalii

AMOUNT_SPIKE Kwota = $10 \times \text{avg_amount}$ karty. Wyzwała regułę <code>LARGE_AMOUNT</code> . $\text{amount} = \text{avg} \times 10$	LOCATION_JUMP Przesunięcie 500–2000 km w 30–180 s. Fizycznie niemożliwa podróż. $\text{prędkość} = \text{dist} / \Delta t > 500$ km/h	HIGH_FREQUENCY 6–12 transakcji w odstępach 50–250 ms. Symulacja card testing attack. $0 < \Delta t < 10$ s
--	---	--

5. Infrastruktura i uruchomienie systemu

Całe środowisko uruchamiane jest przy pomocy Docker Compose. Rozwiązanie wykorzystuje siedem kontenerów obejmujących Apache Kafka, ZooKeeper, Apache Flink, MongoDB oraz narzędzia administracyjne umożliwiające monitorowanie działania systemu.

Po uruchomieniu infrastruktury operator uruchamia generator transakcji, następnie aplikację detekcji fraudów oraz monitor alertów. W trakcie działania

system automatycznie analizuje napływające dane i zapisuje wykryte zdarzenia w bazie MongoDB.

Zastosowanie kontenerów Docker upraszcza proces instalacji, zapewnia powtarzalność środowiska oraz umożliwia łatwe przenoszenie rozwiązania pomiędzy komputerami.

6. Podsumowanie

Zrealizowany system spełnia wszystkie wymagania projektowe dotyczące wykrywania anomalii w transakcjach kartowych. Rozwiązanie wykorzystuje nowoczesne technologie przetwarzania strumieniowego i umożliwia analizę danych w czasie zbliżonym do rzeczywistego.

Projekt obejmuje generator danych, mechanizm wykrywania fraudów, system alarmowy oraz warstwę wizualizacyjną. Dzięki zastosowaniu Apache Flink możliwe jest utrzymywanie stanu każdej karty i wykonywanie analiz historycznych bez utraty wydajności. System może stanowić podstawę do dalszego rozwoju, między innymi poprzez zastosowanie modeli uczenia maszynowego lub bardziej zaawansowanych metod analizy zachowań użytkowników.

Wyniki testów oprogramowania:

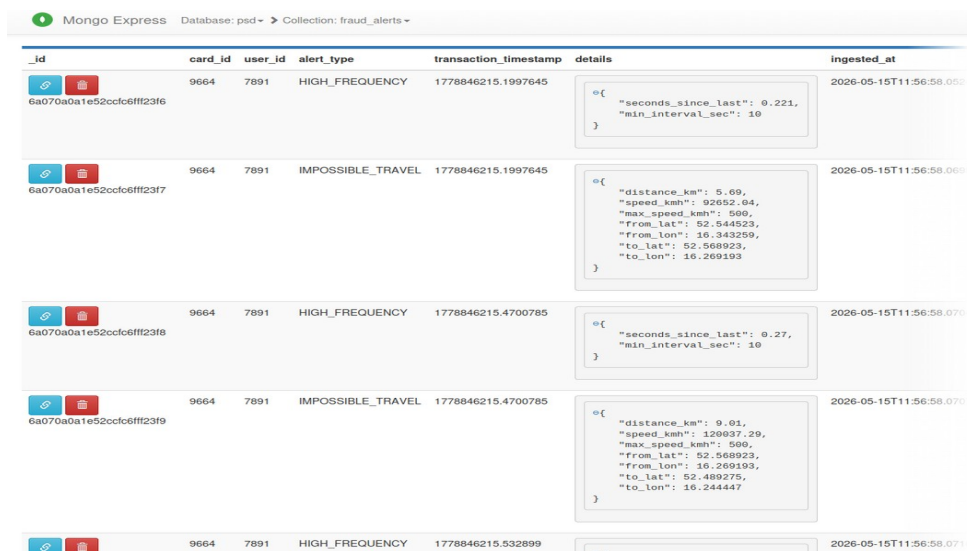
System przetestowano end-to-end w środowisku lokalnym uruchomionym przez Docker Compose (Kafka, Flink, MongoDB oraz narzędzia administracyjne). Po starcie infrastruktury uruchomiono kolejno generator transakcji, detektor anomalii (PyFlink) oraz oba konsumenty: wizualizator strumienia transactions i monitor alertów z tematu alerts. Test potwierdził poprawny przepływ danych od symulatora przez Kafkę do detektora, a następnie do MongoDB i kanału powiadomień.

W trakcie testu detektor wygenerował alerty wszystkich trzech typów: LARGE_AMOUNT (kwota przekraczająca 3× średnią kroczącą wydatków karty), IMPOSSIBLE_TRAVEL (obliczona prędkość przemieszczania powyżej 500 km/h) oraz HIGH_FREQUENCY (kolejna transakcja w odstępie krótszym niż 10 s). Monitor alertów prezentował je w czasie rzeczywistym w formie banerów ASCII i

paneli Rich, z czytelnym uzasadnieniem reguły. Wizualizator transakcji pokazywał ciągły strumień operacji (~5–10 tx/s) w tabeli aktualizowanej na żywo. W Mongo Express zweryfikowano zapis rekordów w kolekcji psd.fraud_alerts (w tym pole ingested_at), a w Kafka UI — obecność tematów transactions i alerts oraz ruch wiadomości. Flink Web UI potwierdził dostępność klastra i możliwość obserwacji metryk jobów.

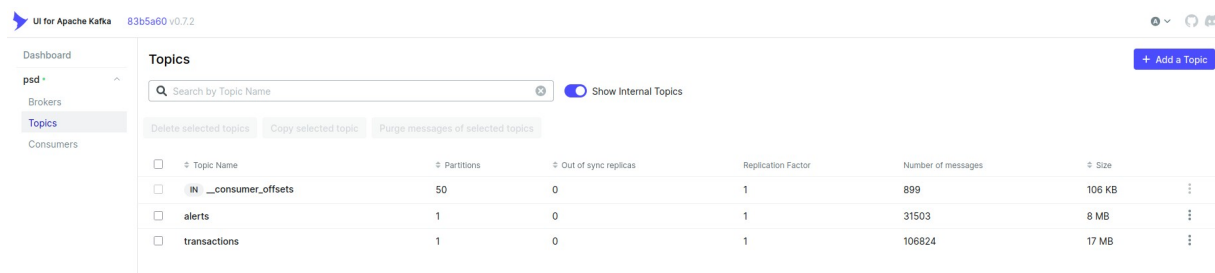
Wyniki testów potwierdzają, że system spełnia założenia projektowe: symuluje realistyczny ruch transakcyjny z kontrolowanymi anomaliami, wykrywa je regułami strumieniowymi w czasie zbliżonym do rzeczywistego, persystuje alerty i przekazuje je operatorowi przez dedykowany kanał. Poniżej przedstawiono zrzuty ekranu interfejsów webowych (:8081 Flink, :8082 Mongo Express, :8083 Kafka UI) oraz terminali konsumentów wizualizujących strumień transakcji i alertów fraudowych.

Kolekcja fraud_alerts w mongodb:



_id	card_id	user_id	alert_type	transaction_timestamp	details	ingested_at
6a070a0a1e52ccfc6fff23f6	9664	7891	HIGH_FREQUENCY	1778846215.1997645	<pre>{ "seconds_since_last": 0.221, "min_interval_sec": 10 }</pre>	2026-05-15T11:56:58.050
6a070a0a1e52ccfc6fff23f7	9664	7891	IMPOSSIBLE_TRAVEL	1778846215.1997645	<pre>{ "distance_km": 5.69, "speed_kmh": 92652.04, "max_speed_kmh": 500, "from_lat": 52.544523, "from_lon": 16.343259, "to_lat": 52.568923, "to_lon": 16.269193 }</pre>	2026-05-15T11:56:58.060
6a070a0a1e52ccfc6fff23f8	9664	7891	HIGH_FREQUENCY	1778846215.4700785	<pre>{ "seconds_since_last": 0.27, "min_interval_sec": 10 }</pre>	2026-05-15T11:56:58.070
6a070a0a1e52ccfc6fff23f9	9664	7891	IMPOSSIBLE_TRAVEL	1778846215.4700785	<pre>{ "distance_km": 9.01, "speed_kmh": 120037.29, "max_speed_kmh": 500, "from_lat": 52.568923, "from_lon": 16.269193, "to_lat": 52.489275, "to_lon": 16.244447 }</pre>	2026-05-15T11:56:58.070
	9664	7891	HIGH_FREQUENCY	1778846215.532899	<pre>{ "seconds_since_last": 0.27, "min_interval_sec": 10 }</pre>	2026-05-15T11:56:58.070

Topiki Apache Kafka:



Topic Name	Partitions	Out of sync replicas	Replication Factor	Number of messages	Size
_consumer_offsets	50	0	1	899	106 KB
alerts	1	0	1	31503	8 MB
transactions	1	0	1	106824	17 MB

Wizualizacja transakcji odbieranych przez konsumenta:

Kafka · transactions							
Time	Card	User	Amount	Limit	Lat	Lon	Flag
19:33:22	5221	830	128.29	12,817.01	52.4401	20.4580	ALERT
19:33:22	763	4792	138.80	3,873.71	49.9904	19.2339	
19:33:23	686	2481	289.28	2,741.98	49.2191	19.0118	
19:33:23	5489	3079	77.89	2,661.44	51.6686	21.8900	
19:33:23	2499	145	188.17	10,267.16	50.9839	17.2622	
19:33:23	6172	2520	304.96	5,748.18	54.7369	21.4800	
19:33:23	3834	1890	312.60	13,727.74	54.1703	18.4079	
19:33:23	2869	7479	50.69	5,348.34	49.9289	20.4533	
19:33:23	9858	1502	149.85	10,896.24	53.0789	20.1988	
19:33:23	582	3779	88.23	5,965.22	49.8616	15.8607	
19:33:23	4532	5907	77.52	8,208.33	49.8890	17.3769	
19:33:23	5677	6958	26.98	3,086.81	51.4825	23.0313	
19:33:23	7237	4179	1,875.50	4,836.23	53.6472	18.2607	
19:33:23	8111	4630	70.44	2,415.08	51.7774	16.0462	
19:33:23	120	2486	135.44	3,825.26	51.1518	19.2277	
19:33:23	2829	3372	63.71	3,980.56	50.6012	15.5771	
19:33:23	682	5996	129.89	12,094.59	52.6700	23.6892	
19:33:23	7292	2917	121.25	7,438.59	52.4097	14.3664	
19:33:23	8180	6930	108.06	2,607.68	52.4270	22.8669	
19:33:23	4277	3669	27.22	2,988.09	50.3196	22.9370	

Wizualizacja alertów fraudowych:

ALERT

ALERT FRAUDOWY

Karta (card id):1630

Typ anomalii:Nieemożliwa podróż (IMPOSSIBLE_TRAVEL)

Czas zdarzenia:2026-05-29 19:29:35

Uzasadnienie:Prędkość: 630.3 km/h, dystans: 14.88 km

user_id=753

ALERT

ALERT FRAUDOWY

Karta (card id):9668

Typ anomalii:Wysoka kwota (LARGE_AMOUNT)

Czas zdarzenia:2026-05-29 19:29:36

Uzasadnienie:Kwota 1703.3 PLN > 3× średnia (104.41 PLN), próg: 313.23 PLN

user_id=7073